Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Hafta 12: Eşzamanlılık ve Ağ Temelleri

Yazılım Geliştirme II – Çalışma Kağıdı

Ad Soyad: _____	**Numara:** _____	**Tarih:** _____

Yönergeler

- Bu çalışma kağıdı **5 bölümden** oluşmaktadır.
- Çoktan seçmeli sorularda doğru seçeneği **daire içine alın**.
- Boşluk doldurma sorularında cevabınızı **çizgi üzerine** yazın.
- Doğru/yanlış sorularında **D** veya **Y** yazın.
- Süre: **30 dakika**
- Toplam: **60 puan**
- Ders kodu: **BILB2000** – 2026 Bahar

Bölüm A: Çoktan Seçmeli Sorular

1. Bir thread-safe queue tasarımında **arayüz yarışı** (interface race) ne anlama gelir? **2 puan**

   A) İki thread aynı anda push çağırdığında veri bozulması
   B) empty, front ve pop ayrı ayrı kilitlenmiş olsalar bile bu üç çağrı arasında başka thread durumu değiştirebilir; mantıksal yarış oluşur
   C) Compiler'ın metod adlarını yanlış sıralaması
   D) std::queue'nun template parametresinin yanlış seçilmesi

2. OBYS sunucunuzda öğrenci kayıtlarını tutan bir std::map var; saatte bir kez yazılıyor ama yüzlerce thread tarafından sürekli okunuyor. Hangi kilit yapısı en uygundur? **2 puan**

   A) std::mutex – en hızlı seçenektir
   B) std::shared_mutex – birden çok okuyucu paralel kilit alabilir, yazıcı yalnız çalışır
   C) std::atomic<std::map> – atomic koruma map için yeterlidir
   D) Hiç kilit kullanma, std::map thread-safe'tir

3. TCP üç-yollu el sıkışması (three-way handshake) sırasıyla hangi paket sırasıyla gerçekleşir? **2 puan**

   A) ACK → SYN → SYN-ACK
   B) SYN → ACK → FIN
   C) SYN → SYN-ACK → ACK
   D) PUSH → ACK → FIN-ACK

4. POSIX soket programlamada `htons(8080)` çağrısı neden gereklidir? **2 puan**

   A) Port numarasını şifrelemek için
   B) Port numarasını host byte order'dan network byte order'a (big-endian) çevirmek için
   C) Port numarasını string'e dönüştürmek için
   D) Yalnızca Windows sistemlerinde gereklidir, Linux'ta gereksizdir

5. TCP **stream** yapısı gereği `recv(s, buf, 1024, 0)` çağrısı 1024 byte istemesine rağmen 50 byte alabilir. Bu durumda doğru tasarım yaklaşımı nedir? **2 puan**

   A) Daha büyük bir buffer kullanmak; eninde sonunda hepsi gelir
   B) UDP'ye geçmek; o paket-tabanlıdır
   C) Mesaj sınırlarını korumak için **çerçeveleme** (örneğin length-prefix) uygulamak ve gelen byte'ları akümülatöre biriktirmek
   D) `recv` yerine `read` kullanmak

Bölüm B: Boşluk Doldurma

Aşağıdaki cümlelerde boş bırakılan yerleri uygun ifadelerle doldurun.

1. OSI katman modelinde uygulama düzeyinde HTTP, FTP gibi protokoller _____ katmanında; TCP ve UDP _____ katmanında; IP ise _____ katmanında yer alır. **2 puan**
2. Qt sinyal/slot mekanizmasının çalışabilmesi için sınıf tanımının başında _____ makrosu bulunmalı ve build sistemi (qmake/CMake) _____ (Meta Object Compiler) aracını çalıştırarak meta-bilgi kodunu üretmelidir. **2 puan**
3. Aynı portu kullanan bir sunucuyu kapatıp hemen yeniden başlatmak istediğinizde işletim sistemi soketi `TIME_WAIT` durumunda tutar. Bu sorunu aşmak için soket seçeneklerinde `SO_`_____ kullanılır; aksi halde `bind` çağrısı `Address already in use` hatası döner. **2 puan**

Bölüm C: Doğru / Yanlış

Aşağıdaki ifadelerin doğru mu yanlış mı olduğunu belirtin.

1. UDP bağlantısız (connectionless) bir protokoldür; paket sırası ve teslim garantisi vermez, ancak gecikme açısından TCP'ye göre genellikle daha düşüktür. **2 puan**
2. QTcpSocket::waitForReadyRead çağrısı GUI thread'inde kullanılırsa olay döngüsünü bloklar; bu nedenle production kodunda readyRead sinyalinin slot'a bağlanması tercih edilmelidir. **2 puan**
3. Length-prefix çerçeveleme yönteminde alıcı taraf bir akümülatör buffer'ında byte'ları biriktirir; her okumada önce uzunluk başlığını çözer, ardından tam mesaj birikene kadar bekler. **2 puan**
4. Sharded ConcurrentMap'te farklı bölmelere düşen iki anahtarın güncellemesi tek bir transaction içinde atomik olarak yapılır; bu yapı ACID garantisi sunar. **2 puan**

Bölüm D: Kısa Cevap

1. Paralel CSV import işleminde dosyadaki satırların **orijinal sırasının korunması** neden önemli olabilir? 2-3 cümle ile açıklayın. **6 puan**

2. Bir thread havuzunun (thread pool) kuyruğunu **bounded** (kapasitesi sınırlı) yapmak hangi soruna çözüm sağlar? Üreticinin (producer) bu kuyruğa eleman eklerken kuyruk doluysa ne olur? **6 puan**

Bölüm E: Kod Analizi

1. Aşağıdaki POSIX TCP istemci kodunda iki kritik hata vardır. Hataları bulun ve nasıl düzelteceğinizi açıklayın. **12 puan**

```
int s = socket(AF_INET, SOCK_STREAM, 0);

sockaddr_in adres{};
adres.sin_family = AF_INET;
adres.sin_port = 8080;                          // (1)
adres.sin_addr.s_addr = inet_addr("127.0.0.1");

connect(s, (sockaddr*)&adres, sizeof(adres));

const char* mesaj = "Merhaba sunucu";
send(s, mesaj, 14, 0);

char buf[1024];
int n = recv(s, buf, 1024, 0);
std::cout << "Yanit: " << buf << std::endl;      // (2)

close(s);
```

Bölüm F: Senaryo Bazlı Tasarım

1. OBYS sunucunuza eş zamanlı olarak yaklaşık **100 istemcinin** bağlanması bekleniyor; her istemci JSON formatında öğrenci sorgusu gönderip cevap alacak. Aşağıdaki üç tasarım yaklaşımından hangisini seçerdiniz ve **neden**? Cevabınızı 3-4 cümleyle gerekçelendirin. **12 puan**

   - **(a)** Her istemciye bir std::thread (thread-per-client)
   - **(b)** Sabit boyutlu bir thread havuzu (örneğin 16 worker) ve thread-safe iş kuyruğu

- **(c)** Tek thread + olay tabanlı I/O multiplexing (`select`/`epoll` veya Qt'nin asenkron sinyalleri)

Başarılar!